

Question 1: What is a function in Python? Explain how functions are created and called. Also explain parameters, arguments, default parameters, *args, and **kwargs with suitable examples.

Answer :- A function in Python is a reusable block of code designed to perform a specific task. Functions help organize code, improve readability, and prevent repetition (DRY — Don't Repeat Yourself principle).

1. Creating and Calling Functions

- **Creating a function:** Functions are defined using the `def` keyword, followed by the function name, parentheses `()`, and a colon `:`. The code block inside the function is indented.
- **Calling a function:** To execute the function, write its name followed by parentheses `()`.

Python

```
# Creating (defining) a function
```

```
def greet():
```

```
    print("Hello, welcome to Python learning!")
```

```
# Calling the function
```

```
greet()
```

2. Parameters vs. Arguments

- **Parameters:** Variables listed inside the parentheses in the function definition.
- **Arguments:** The actual values sent to the function when it is called.

Python

'name' is a parameter

```
def welcome_user(name):
```

```
    print(f"Welcome, {name}!")
```

'Rahul' is an argument

```
welcome_user("Rahul")
```

3. Default Parameters

A default parameter provides a fallback value if no argument is passed during the function call.

Python

```
def greet_person(name="Guest"):
```

```
    print(f"Hello, {name}!")
```

```
greet_person("Amit") # Output: Hello, Amit!
```

```
greet_person()      # Output: Hello, Guest! (uses default value)
```

4. `*args` (Arbitrary Positional Arguments)

`*args` allows a function to accept any number of **positional arguments**. Inside the function, `args` becomes a tuple containing all passed values.

Python

```
def calculate_sum(*args):
```

```
    total = sum(args)
```

```
    print("Sum:", total)
```

```
calculate_sum(10, 20)      # Output: Sum: 30
```

```
calculate_sum(5, 10, 15, 20)  # Output: Sum: 50
```

5. `**kwargs` (Arbitrary Keyword Arguments)

`**kwargs` allows a function to accept any number of **keyword (named) arguments**. Inside the function, `kwargs` becomes a dictionary containing key-value pairs.

Python

```
def display_student_info(**kwargs):
```

```
for key, value in kwargs.items():
```

```
    print(f"{key}: {value}")
```

```
display_student_info(Name="Rahul", Age=21, Course="Data  
Science")
```

```
# Output:
```

```
# Name: Rahul
```

```
# Age: 21
```

```
# Course: Data Science
```

Question 2: Create a Python program with a Person class containing the person's name and a method to display it. Then create a Student class that inherits from Person and adds the student's course. Create an object of Student and display both details.

Answer :- Python

Parent Class

```
class Person:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def display_person(self):
```

```
        print(f"Name: {self.name}")
```

Child Class inheriting from Person

```
class Student(Person):
```

```
    def __init__(self, name, course):
```

```
        # Call the constructor of the parent class (Person)
```

```
        super().__init__(name)
```

```
self.course = course
```

```
def display_student_details(self):  
    # Display name using parent class method  
    self.display_person()  
    # Display course  
    print(f"Course: {self.course}")
```

```
# Creating an object of the Student class
```

```
student1 = Student("Rahul Sharma", "Data Science")
```

```
# Displaying both details
```

```
print("--- Student Details ---")
```

```
student1.display_student_details()
```

Output

Plaintext

```
--- Student Details ---
```

Name: Rahul Sharma

Course: Data Science

Explanation

1. **Person Class (Parent Class):** Takes `name` in its constructor (`__init__`) and provides `display_person()` to output the name.
2. **Student Class (Child Class):** Inherits from `Person` using `class Student(Person):`. It invokes `super().__init__(name)` to pass the name parameter to the parent class and adds its own `course` attribute.
3. **Execution:** An instance of `Student` is created with name `"Rahul Sharma"` and course `"Data Science"`. Calling `display_student_details()` outputs both the inherited and child class attribute

Question 3: What is variable scope in Python? Explain local and global variables. Why should excessive use of the global keyword be avoided? Also explain nested functions and closures.

Answer :- **Variable scope** refers to the region of a Python program where a variable is defined and accessible. Scope determines where in your code a particular variable can be read or modified. Python follows the **LEGB rule** for looking up names: **L**ocal, **E**nclosing, **G**lobal, and **B**uilt-in.

2. Local vs. Global Variables

- **Local Variables:** Variables defined inside a function. They are created when the function is called and destroyed when the function finishes execution. They cannot be accessed outside the function.
- **Global Variables:** Variables defined outside any function, usually at the main body/top level of the script. They are accessible from anywhere within the file.

Python

```
x = "Global Variable" # Global variable
```

```
def my_function():
```

```
    y = "Local Variable" # Local variable
```

```
    print("Inside function:", x) # Accessing global
```

```
    print("Inside function:", y) # Accessing local
```

```
my_function()
```

Trying to access y outside the function will raise a NameError:

print(y) # NameError: name 'y' is not defined

3. Why Should Excessive Use of the `global` Keyword Be Avoided?

The `global` keyword allows you to modify a global variable from inside a function. However, using it excessively is considered bad programming practice for several reasons:

1. **Harder to Debug:** Global variables can be modified from anywhere in the codebase, making it difficult to trace where bugs or unintended mutations originate.
2. **Loss of Function Isolation:** Functions become tightly coupled to external state, making them harder to reuse in other parts of a project or test independently.
3. **Concurrency Issues:** In multi-threaded or multi-process applications, modifying global variables can lead to race conditions and unexpected behavior.
4. **Poor Code Structure:** Passing inputs via function arguments and returning outputs explicitly makes code much clearer and more maintainable.

4. Nested Functions and Closures

Nested Functions

A **nested function** (or inner function) is simply a function defined inside another function. The inner function can read variables defined in the outer (enclosing) function's scope.

Python

```
def outer_function(msg):
```

```
# Enclosing scope variable
```

```
def inner_function():
```

```
    # Accessing outer variable
```

```
    print(f"Message: {msg}")
```

```
inner_function()
```

```
outer_function("Hello from the inner function!")
```

Closures

A **closure** occurs when a nested function remembers and accesses variables from its enclosing scope even after the outer function has finished executing and returned.

For a function to be a closure:

1. There must be a nested function.
2. The nested function must refer to a variable defined in the enclosing function.
3. The enclosing function must return the nested function object.

Python

```
def multiplier(factor):
```

```
    def multiply(number):
```

```
# 'factor' is remembered from the outer scope

return number * factor

return multiply # Returns the inner function object without calling it


# Creating closures

double = multiplier(2)

triple = multiplier(3)


print(double(5)) # Output: 10

print(triple(5)) # Output: 15
```

In the example above, `double` and `triple` retain access to their respective `factor` values even though `multiplier()` has completed execution.

Question 4: What is a lambda function? Explain how `map()`, `filter()`, and `reduce()` work in Python with suitable examples. A

Answer :- lambda function in Python is a small, anonymous function (a function defined without a name). It is defined using the `lambda` keyword and can take any number of arguments, but can only contain a single expression.

Syntax:

Python

lambda arguments: expression

Example:

Python

Regular function

```
def add(x, y):
```

```
    return x + y
```

Equivalent lambda function

```
add_lambda = lambda x, y: x + y
```

```
print(add_lambda(5, 3)) # Output: 8
```

2. `map()` Function

The `map()` function applies a given function to every item of an iterable (like a list) and returns a map object (an iterator).

Syntax:

Python

```
map(function, iterable)
```

Example:

Python

```
numbers = [1, 2, 3, 4, 5]
```

```
# Using map() with a lambda function to double each number
```

```
doubled = list(map(lambda x: x * 2, numbers))
```

```
print(doubled) # Output: [2, 4, 6, 8, 10]
```

3. `filter()` Function

The `filter()` function constructs an iterator from elements of an iterable for which a function returns `True`. It filters out items that do not meet the condition.

Syntax:

Python

```
filter(function, iterable)
```

Example:

Python

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
# Using filter() with a lambda function to keep only even numbers
```

```
evens = list(filter(lambda x: x % 2 == 0, numbers))
```

```
print(evens) # Output: [2, 4, 6, 8, 10]
```

4. `reduce()` Function

The `reduce()` function continuously applies a function to pairs of elements in an iterable, reducing the entire sequence down to a single cumulative value.

Note: In Python 3, `reduce()` must be imported from the `functools` module.

Syntax:

Python

```
from functools import reduce
```

```
reduce(function, iterable)
```

Example:

Python

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4, 5]
```

```
# Using reduce() with a lambda function to calculate the product of all  
elements
```

```
product = reduce(lambda x, y: x * y, numbers)
```

```
print(product) # Output: 120 (((((1 * 2) * 3) * 4) * 5)
```

Question 5: Look at the code below and answer the questions:

Python

```
class Parent:
```

```
    def show(self):
```

```
        print("Parent")
```

```
class Child(Parent):
```

```
    def show(self):
```

```
        print("Child")
```

```
obj = Child()
```

```
obj.show()
```

What will be printed? Why is the Parent version of `show()` not called?

Answer :- Plaintext

Child

2. Why is the Parent version of `show()` not called?

This happens because of **Method Overriding** in Object-Oriented Programming (OOP):

1. **Method Overriding:** The `Child` class defines its own `show()` method with the exact same name as the `show()` method in the `Parent` class.
2. **Dynamic Resolution:** When `obj.show()` is executed, Python checks the object's class (`Child`) first to see if it has a `show()` method.
3. **Shadowing:** Since `show()` exists in `Child`, Python executes that version directly. It stops searching further up the inheritance hierarchy, so the `Parent` class version is overridden (shadowed) and never called.

Question 6: Create a custom iterator class called `EvenNumbers` that generates even numbers from 2 to 10. Use `__iter__()`, `__next__()`, and `StopIteration`

Answer :- Python

```
class EvenNumbers:
```

```
    def __init__(self):
```

```
        # Initialize starting even number
```

```
        self.current = 2
```

```
    def __iter__(self):
```

```
        # An iterator must return itself
```

```
        return self
```

```
    def __next__(self):
```

```
        # Check if current value exceeds limit (10)
```

```
        if self.current <= 10:
```

```
            val = self.current
```

```
            self.current += 2 # Increment by 2 for next even number
```

```
            return val
```

```
        else:
```

```
            # Raise StopIteration when sequence ends
```

```
raise StopIteration
```

```
# Using the custom iterator with a for loop
```

```
even_iter = EvenNumbers()
```

```
print("Even numbers from 2 to 10:")
```

```
for num in even_iter:
```

```
    print(num)
```

Output

Plaintext

Even numbers from 2 to 10:

2

4

6

8

10

Explanation

1. **`__init__()` Method:** Sets the starting value (`self.current = 2`).
2. **`__iter__()` Method:** Makes the class an iterable object by returning `self`.
3. **`__next__()` Method:** Returns the current even number and increments it by 2 for subsequent calls. Once `self.current` exceeds 10, it raises the `StopIteration` exception, signaling the loop to terminate cleanly.

Question 7: Write a Python program using multiple inheritance. Create two classes, Teacher and Coder, with one method in each. Create a Trainer class that inherits from both and use an object of Trainer to call both methods.

Answer :- Python

Parent Class 1

class Teacher:

def teach(self):

print("Teaching students core concepts and theory.")

Parent Class 2

class Coder:

```
def code(self):

    print("Writing and optimizing Python code.")


# Child Class inheriting from both Teacher and Coder (Multiple Inheritance)

class Trainer(Teacher, Coder):

    def train(self):

        print("Trainer conducts complete training sessions.")


# Creating an object of Trainer

trainer_obj = Trainer()


# Calling methods inherited from both parent classes

print("--- Calling Methods via Trainer Object ---")

trainer_obj.teach() # Method from Teacher class

trainer_obj.code() # Method from Coder class
```

Output

Plaintext

--- Calling Methods via Trainer Object ---

Teaching students core concepts and theory.

Writing and optimizing Python code.

Explanation

1. **Multiple Inheritance:** The `Trainer` class inherits from two parent classes at the same time: `class Trainer(Teacher, Coder) :`
2. **Method Access:** Because of multiple inheritance, an object of `Trainer` gets access to all public methods of both `Teacher` (`teach()`) and `Coder` (`code()`).

Question 8: What is polymorphism? Explain it in your own words with a Python example where two different classes have the same method name but perform different tasks.

Answer :- **Explanation**

Polymorphism literally means "many forms" (poly = many, morph = form). In Object-Oriented Programming, polymorphism allows different objects to respond to the same method name in their own unique way.

This means you can call the same method on objects of completely different classes without worrying about which class the object belongs to—each object knows how to perform its own version of that action.

Python Example

Python

Class 1: Dog

class Dog:

def speak(self):

return "Woof! Woof!"

Class 2: Cat

class Cat:

def speak(self):

return "Meow! Meow!"

Function demonstrating polymorphism

def make_animal_speak(animal):

Calling the same method 'speak()' regardless of the class

print(animal.speak())

Creating instances

```
dog = Dog()
```

```
cat = Cat()
```

```
# Executing the method on different class instances
```

```
print("--- Polymorphism in Action ---")
```

```
make_animal_speak(dog) # Output: Woof! Woof!
```

```
make_animal_speak(cat) # Output: Meow! Meow!
```

Output

Plaintext

```
--- Polymorphism in Action ---
```

```
Woof! Woof!
```

```
Meow! Meow!
```

How it works:

1. Both `Dog` and `Cat` classes have a method with the exact same name: `speak()`.
2. The function `make_animal_speak()` takes any object and calls `animal.speak()`.

3. When `dog` is passed, it executes `Dog.speak()`, and when `cat` is passed, it executes `Cat.speak()`.

Question 9: Consider the following list:

numbers = [1, 2, 3, 4, 5, 6] Write a Python program to:

1. Use map() to calculate the square of every number.
2. Use filter() to select only even numbers.
3. Use reduce() to calculate the sum of all numbers.
4. Display all three results.

Answer :- Python

```
from functools import reduce
```

```
# Given list
```

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
# 1. Use map() to calculate the square of every number
```

```
squares = list(map(lambda x: x**2, numbers))
```

```
# 2. Use filter() to select only even numbers
```

```
evens = list(filter(lambda x: x % 2 == 0, numbers))
```

3. Use reduce() to calculate the sum of all numbers

```
total_sum = reduce(lambda x, y: x + y, numbers)
```

4. Display all three results

```
print("Original List:", numbers)
```

```
print("1. Squares of numbers:", squares)
```

```
print("2. Even numbers:", evens)
```

```
print("3. Sum of all numbers:", total_sum)
```

Output

Plaintext

Original List: [1, 2, 3, 4, 5, 6]

1. Squares of numbers: [1, 4, 9, 16, 25, 36]

2. Even numbers: [2, 4, 6]

3. Sum of all numbers: 21

Explanation

1. **map()**: Applies the **lambda x: x**2** function to each item in **numbers**, returning a list of squared values.

2. **filter()**: Applies `lambda x: x % 2 == 0` to check for even numbers and filters out odd numbers.
3. **reduce()**: Imported from `functools`, it accumulates the values using `lambda x, y: x + y` to compute the sum (`1+2+3+4+5+6=21`).

Question 10: You are developing a payment system for an online shopping application. The system should support UPI, credit card, and wallet payments. Each payment type should have its own way of processing the payment, but the main program should be able to call the same `pay()` method for all of them. Design a simple Python solution using inheritance and polymorphism, and explain how it works.

Answer :- Python

```
# Base Class (Parent)
```

```
class PaymentMethod:
```

```
    def pay(self, amount):
```

```
        raise NotImplementedError("Subclasses must implement abstract method pay()")
```

```
# Child Class 1: UPI Payment
```

```
class UPIPayment(PaymentMethod):
```

```
def __init__(self, upi_id):
```

```
    self.upi_id = upi_id
```

```
def pay(self, amount):
```

```
    print(f"Paid ₹{amount} successfully using UPI ID: {self.upi_id}")
```

Child Class 2: Credit Card Payment

```
class CreditCardPayment(PaymentMethod):
```

```
    def __init__(self, card_number):
```

```
        self.card_number = card_number
```

```
def pay(self, amount):
```

```
    # Displaying masked card number for security
```

```
    masked_card = f"**** *{self.card_number[-4:]}"
```

```
    print(f"Paid ₹{amount} successfully using Credit Card:  
{masked_card}")
```

```
# Child Class 3: Wallet Payment
```

```
class WalletPayment(PaymentMethod):
```

```
    def __init__(self, wallet_name):
```

```
        self.wallet_name = wallet_name
```

```
    def pay(self, amount):
```

```
        print(f"Paid ₹{amount} successfully using {self.wallet_name} Wallet")
```

```
# Main Program demonstrating Inheritance and Polymorphism
```

```
def process_checkout(payment_method, amount):
```

```
    # Calls the same pay() method regardless of the payment class type
```

```
    payment_method.pay(amount)
```

```
# Creating payment instances
```

```
upi = UPIPayment("user@paytm")
```

```
credit_card = CreditCardPayment("1234567890123456")
```

```
wallet = WalletPayment("Paytm")
```

```
print("--- Payment System Execution ---")
```

```
process_checkout(upi, 500)
```

```
process_checkout(credit_card, 1200)
```

```
process_checkout(wallet, 250)
```

Output

Plaintext

```
--- Payment System Execution ---
```

```
Paid ₹500 successfully using UPI ID: user@paytm
```

```
Paid ₹1200 successfully using Credit Card: **** * 3456
```

```
Paid ₹250 successfully using Paytm Wallet
```

Explanation

1. Inheritance:

- **PaymentMethod** acts as the parent base class defining the common interface (**pay()** method).

- `UPIPayment`, `CreditCardPayment`, and `WalletPayment` inherit from `PaymentMethod`.

2. Polymorphism:

- Each child class overrides the `pay()` method to define its own specific payment logic.
- The `process_checkout()` function accepts any payment object and calls `payment_method.pay(amount)`. It does not need to know the specific type of payment being processed, demonstrating polymorphic behavior.